EXP NO:01 Date:	Downloading and installing Hadoop; Understanding different Hadoop modes. Startup scripts, Configuration files.

Aim:

To Downloading and installing Hadoop; Understanding different Hadoop modes.Startup scripts, Configuration files.

Procedure:

1. Prerequisites

First, we need to make sure that the following prerequisites are installed:

1. Java 8 runtime environment (JRE): Hadoop 3 requires a Java 8 installation. I prefer using the offline installer.

2. Java 8 development Kit (JDK)

3. To unzip downloaded Hadoop binaries, we should install 7zip.

4. I will create a folder —E:\hadoop-env‖ on my local machine to store downloaded files.

2. Download Hadoop binaries

The first step is to download Hadoop binaries from the official website. The binary package size is about 342 MB.



Figure 1 — Hadoop binaries download link

After finishing the file download, we should unpack the package using 7zip int two steps. First, we should extract the hadoop-3.2.1.tar.gz library, and then, we should unpack the extracted tar file:

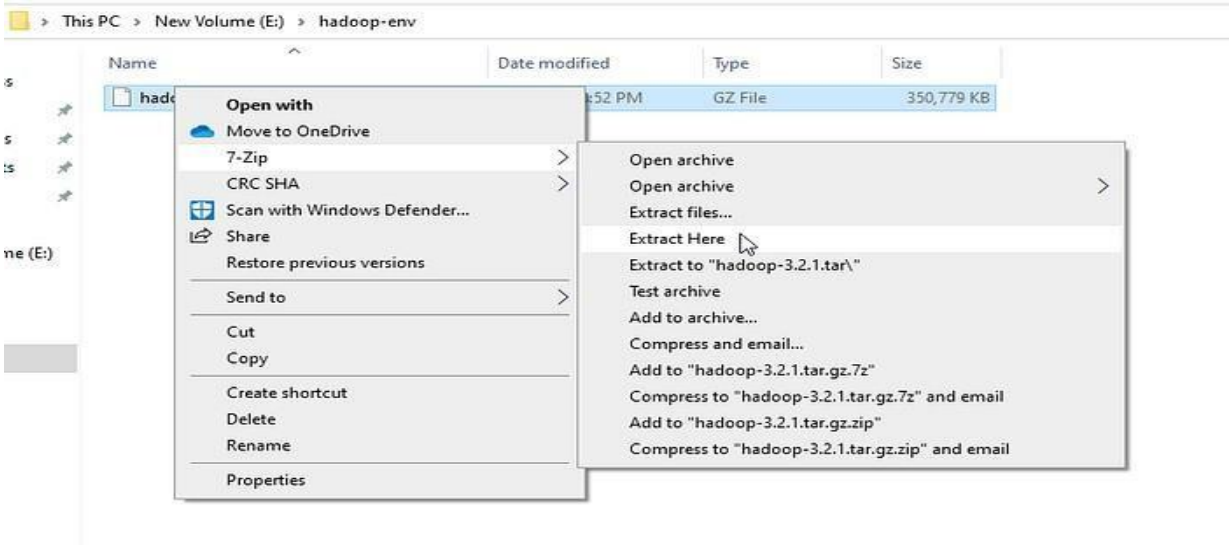


Figure 2 — Extracting hadoop-3.2.1.tar.gz package using 7zip

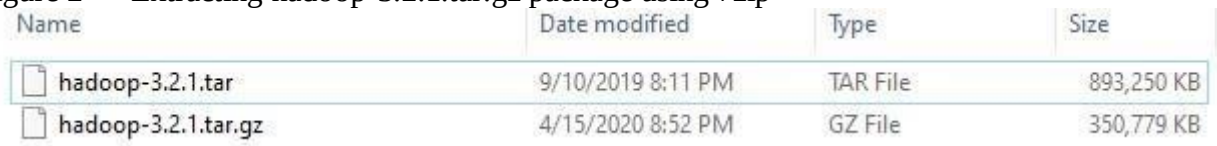


Figure 3 — Extracted hadoop-3.2.1.tar file

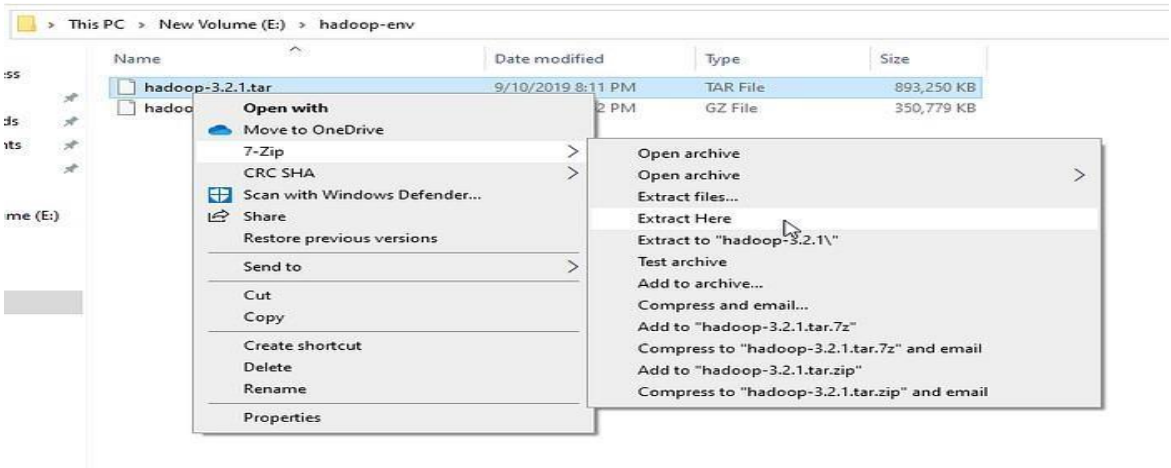


Figure 4 — Extracting the hadoop-3.2.1.tar file

The tar file extraction may take some minutes to finish. In the end, you may see some warnings about symbolic link creation. Just ignore these warnings since they are not related to
windows.

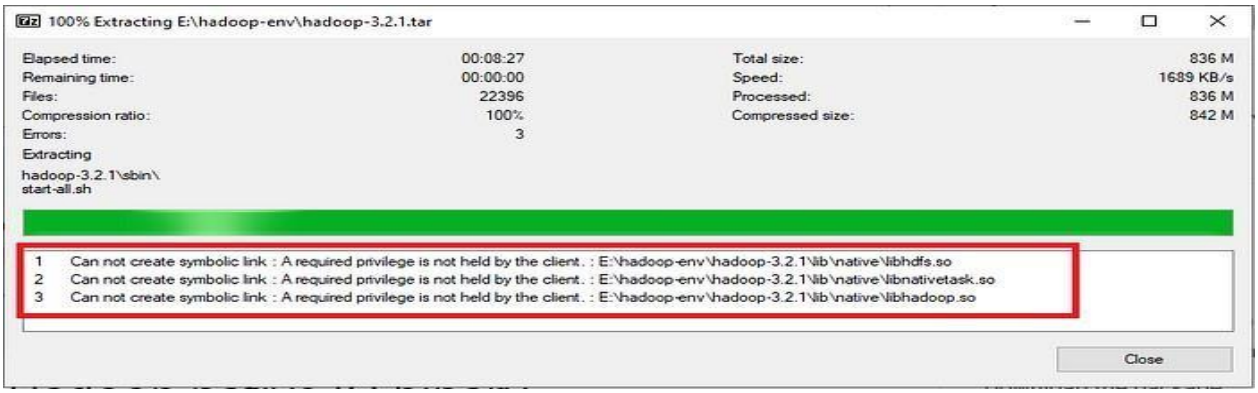


Figure 5 — Symbolic link warnings

After unpacking the package, we should add the Hadoop native IO libraries, which can be found in the following GitHub repository: https://github.com/cdarlint/winutils.

Since we are installing Hadoop 3.2.1, we should download the files located

in https://github.com/cdarlint/winutils/tree/master/hadoop-3.2.1/bin and copy them into the ―hadoop-3.2.1\bin‖ directory.

3. Setting up environment variables

After installing Hadoop and its prerequisites, we should configure the environment variables to define Hadoop and Java default paths.

To edit environment variables, go to Control Panel > System and Security > System (or rightclick > properties on My Computer icon) and click on the ―Advanced system settings‖ link.

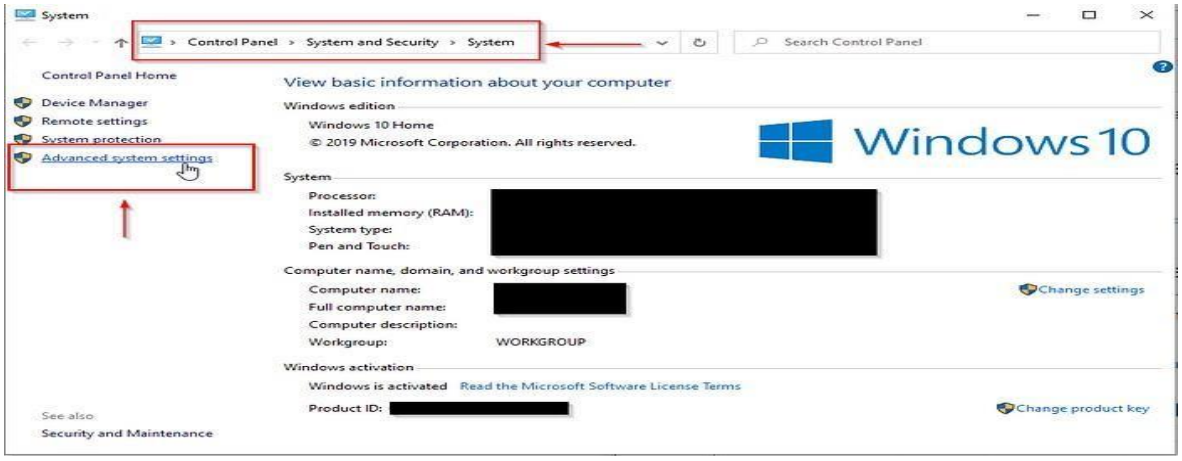


Figure 6 — Opening advanced system settings

When the ―Advanced system settings‖ dialog appears, go to the ―Advanced‖ tab and click on the ―Environment variables‖ button located on the bottom of the dialog.

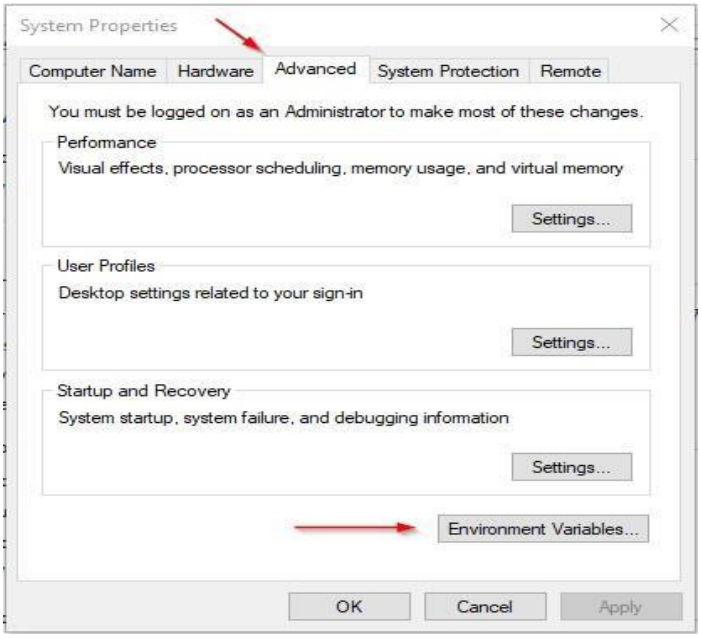


Figure 7 — Advanced system settings dialog

In the —Environment Variables‖ dialog, press the —New‖ button to add a new variable.

Note: In this guide, we will add user variables since we are configuring Hadoop for a single user. If you are looking to configure Hadoop for multiple users, you can define System variables instead. There are two variables to define:

1. JAVA_HOME: JDK installation folder path

2. HADOOP_HOME: Hadoop installation folder path

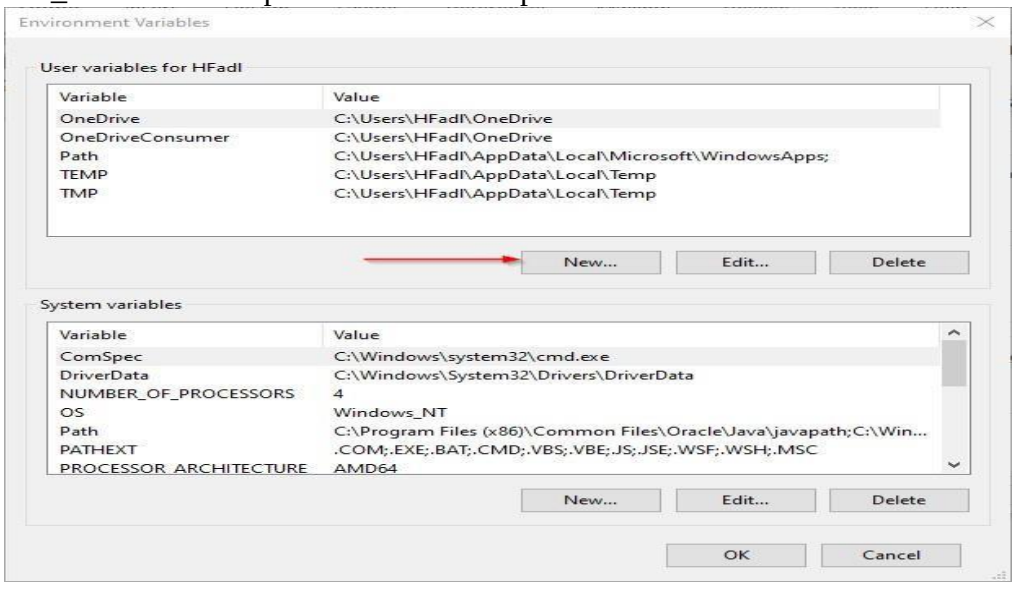


Figure 8 — Adding JAVA_HOME variable

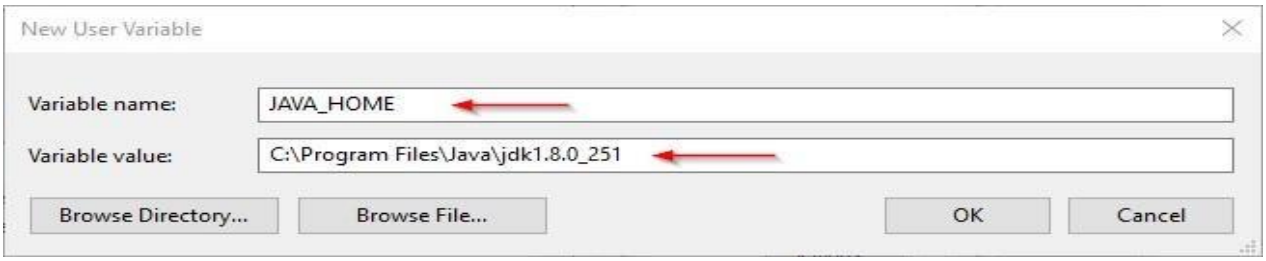


Figure 9 — Adding HADOOP_HOME variable

Now, we should edit the PATH variable to add the Java and Hadoop binaries paths as shown in the following screenshots.



Figure 10 — Editing the PATH variable

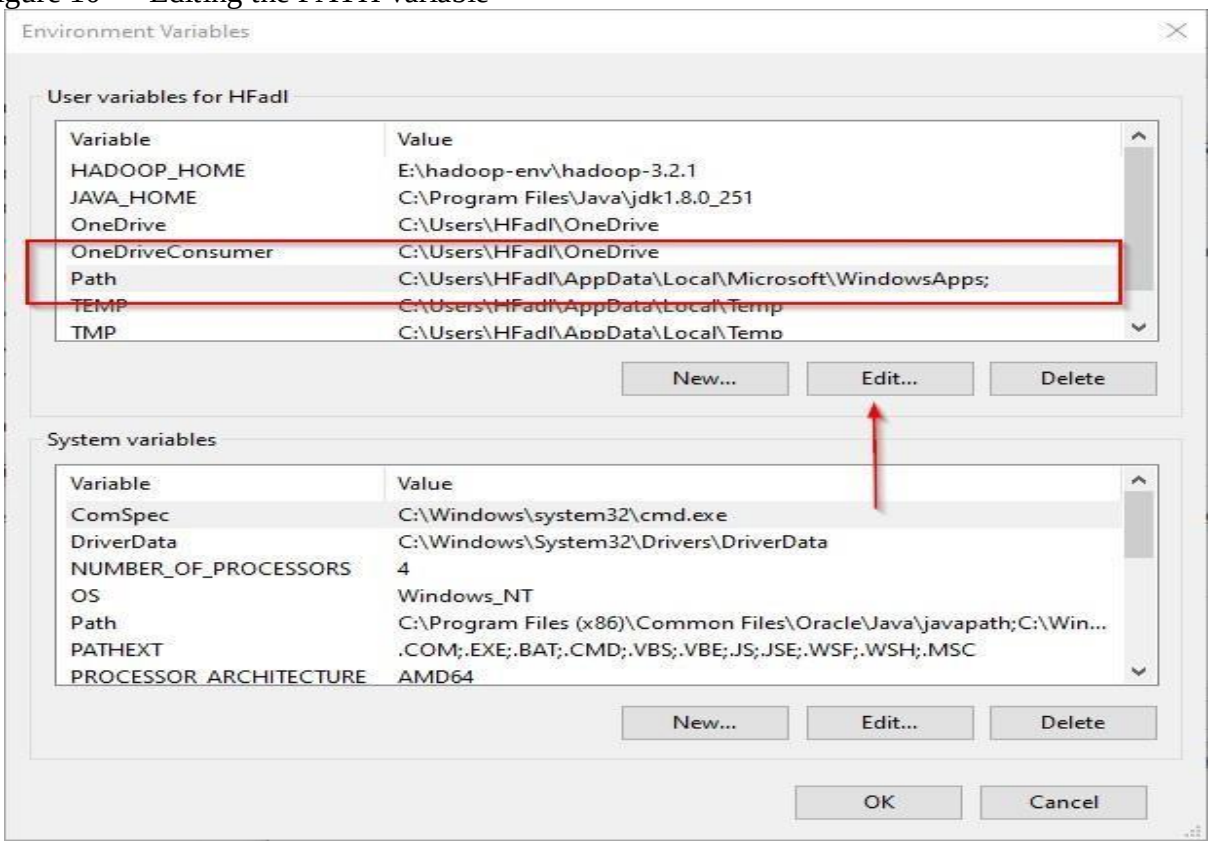


Figure 11 — Editing PATH variable

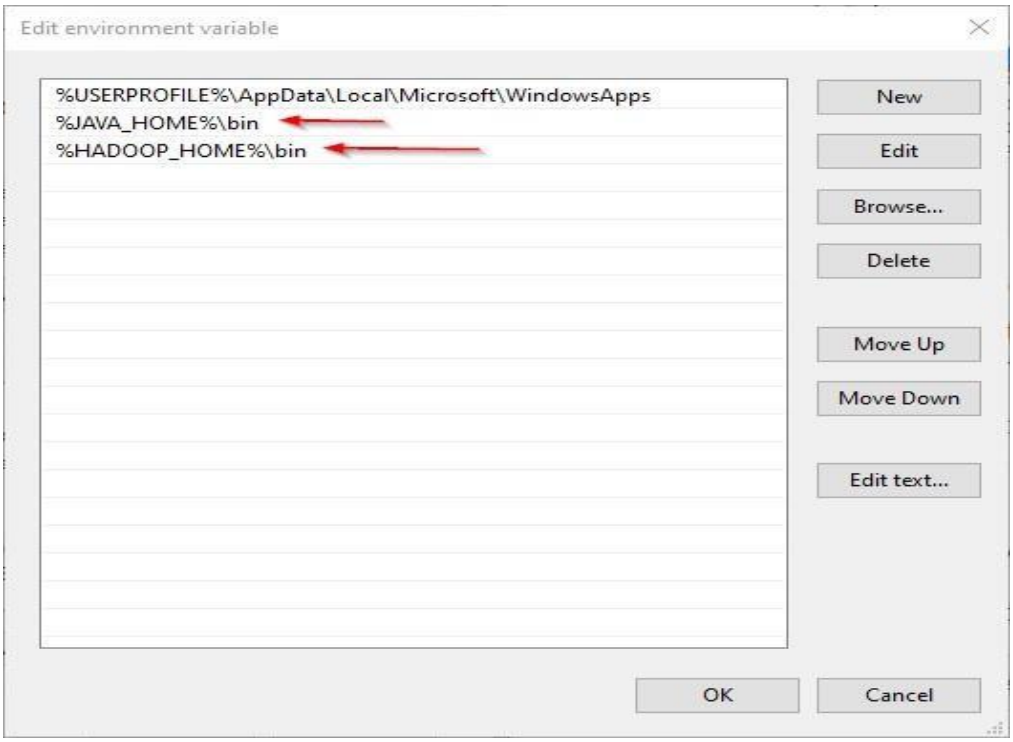


Figure 12— Adding new paths to the PATH variable

3.1. JAVA_HOME is incorrectly set error

Now, let‘s open PowerShell and try to run the following command:

```
hadoop -version
```

In this example, since the JAVA_HOME path contains spaces, I received the following error:

```
JAVA_HOME is incorrectly set
```

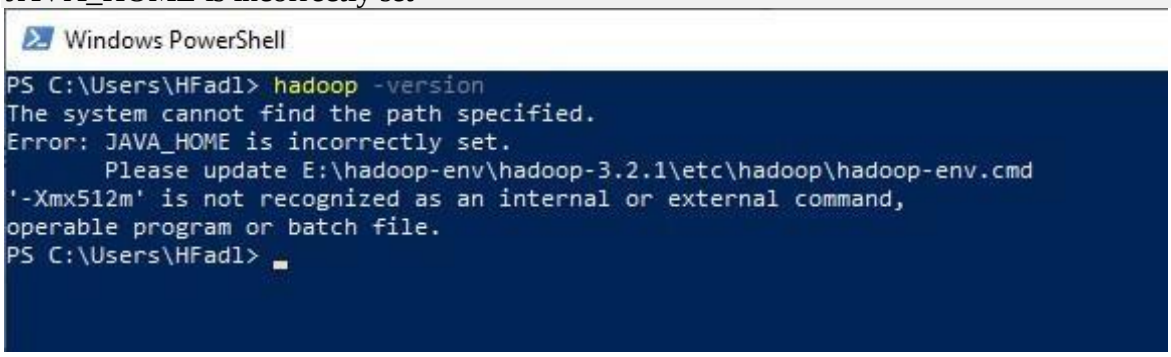


Figure 13 — JAVA_HOME error

To solve this issue, we should use the windows 8.3 path instead. As an example:

- Use —Progra~1‖ instead of —Program Files‖
- Use —Progra~2‖ instead of —Program Files(x86)‖

After replacing —Program Files‖ with —Progra~1‖, we closed and reopened PowerShell and tried the same command. As shown in the screenshot below, it runs without errors.

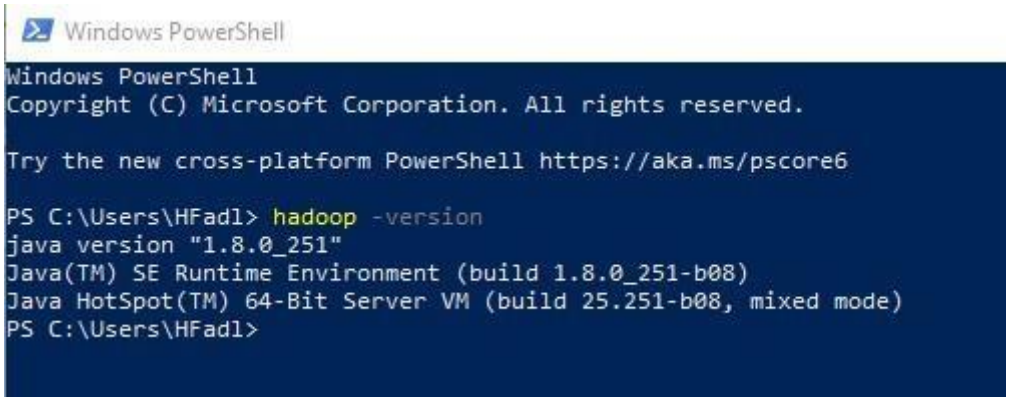


Figure 14 — hadoop -version command executed successfully

4. Configuring Hadoop cluster

There are four files we should alter to configure Hadoop cluster:

1. %HADOOP_HOME%\etc\hadoop\hdfs-site.xml
2. %HADOOP_HOME%\etc\hadoop\core-site.xml
3. %HADOOP_HOME%\etc\hadoop\mapred-site.xml
4. %HADOOP_HOME%\etc\hadoop\yarn-site.xml

4.1. HDFS site configuration

As we know, Hadoop is built using a master-slave paradigm. Before altering the HDFS configuration file, we should create a directory to store all master node (name node) data and another one to store data (data node). In this example, we created the following directories:

- E:\hadoop-env\hadoop-3.2.1\data\dfs\namenode

- E:\hadoop-env\hadoop-3.2.1\data\dfs\datanode

Now, let‘s open —hdfs-site.xml‖ file located in —%HADOOP_HOME%\etc\hadoop‖ directory, and we should add the following properties within the <configuration></configuration> element:

```
<property><name>dfs.replication</name><value>1</value></property><property><name>d
fs.n amenode.name.dir</name><value>file:///E:/hadoop-env/hadoop-
3.2.1/data/dfs/namenode</value></property><property><name>dfs.datanode.data.dir</name
><va lue>file:///E:/hadoop-env/hadoop-3.2.1/data/dfs/datanode</value></property>
```

Note that we have set the replication factor to 1 since we are creating a single node cluster.

4.2. Core site configuration

Now, we should configure the name node URL adding the following XML code into the

<configuration></configuration> element within —core-site.xml‖:

```
<property><name>fs.default.name</name><value>hdfs://localhost:9820</value></property>
```

4.3. Map Reduce site configuration

Now, we should add the following XML code into the <configuration></configuration> element within —mapred-site.xml‖:

```
<property><name>mapreduce.framework.name</name><value>yarn</value><description>
Map Reduce framework name</description></property>
```

4.4. Yarn site configuration

Now, we should add the following XML code into the <configuration></configuration> element within —yarn-site.xml‖:

```
<property><name>yarn.nodemanager.aux-
services</name><value>mapreduce_shuffle</value><description>Yarn Node Manager Aux
Service</description></property>
```

5. Formatting Name node

After finishing the configuration, let‘s try to format the name node using the following command: hdfsnamenode -format

Due to a bug in the Hadoop 3.2.1 release, you will receive the following error: 2020–04–17 22:04:01,503 ERROR namenode.NameNode: Failed to start namenode.java.lang.UnsupportedOperationExceptionatjava.nio.file.Files.setPosixFilePermissions(Files.java:2044)at org.apache.hadoop.hdfs.server.common.Storage$StorageDirectory.clearDirectory(Storage.java:45 2)at org.apache.hadoop.hdfs.server.namenode.NNStorage.format(NNStorage.java:591)at org.apache.hadoop.hdfs.server.namenode.NNStorage.format(NNStorage.java:613)at org.apache.hadoop.hdfs.server.namenode.FSImage.format(FSImage.java:188)at org.apache.hadoop.hdfs.server.namenode.NameNode.format(NameNode.java:1206)at org.apache.hadoop.hdfs.server.namenode.NameNode.createNameNode(NameNode.java:1649)at org.apache.hadoop.hdfs.server.namenode.NameNode.main(NameNode.java:1759)2020–04–17 22:04:01,511 INFO util.ExitUtil: Exiting with status 1: java.lang.UnsupportedOperationException2020–04–17 22:04:01,518 INFO namenode.NameNode: SHUTDOWN_MSG:

This issue will be solved within the next release. For now, you can fix it temporarily using the following steps (reference):

1. Download hadoop-hdfs-3.2.1.jar file from the following link.

2. Rename the file name hadoop-hdfs-3.2.1.jar to hadoop-hdfs-3.2.1.bak in folder

   %HADOOP_HOME%\share\hadoop\hdfs

3. Copy the downloaded hadoop-hdfs-3.2.1.jar to folder

   %HADOOP_HOME%\share\hadoop\hdfs

Now, if we try to re-execute the format command (Run the command prompt or PowerShell as administrator), you need to approve file system format.

Output

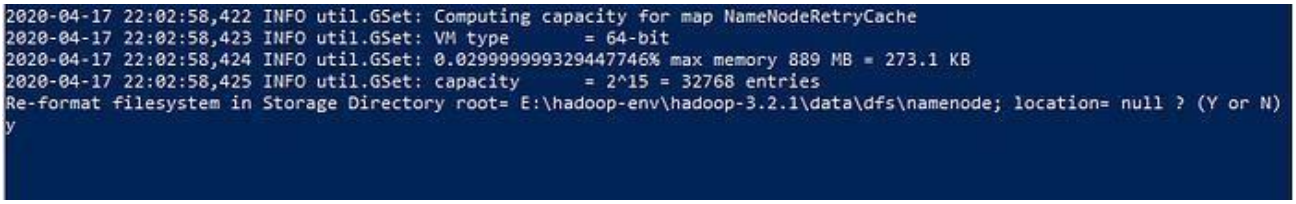


Figure 15 — File system format approval

And the command is executed successfully:

And the command is executed successfully:



Figure 16 — Command executed successfully

6. Starting Hadoop services

Now, we will open PowerShell, and navigate to —%HADOOP_HOME%\sbin‖ directory.
Then we will run the following command to start the Hadoop nodes:

```
.\start-dfs.cmd
```



Figure 17 — StartingHadoop nodes

Two command prompt windows will open (one for the name node and one for the data node) as follows:

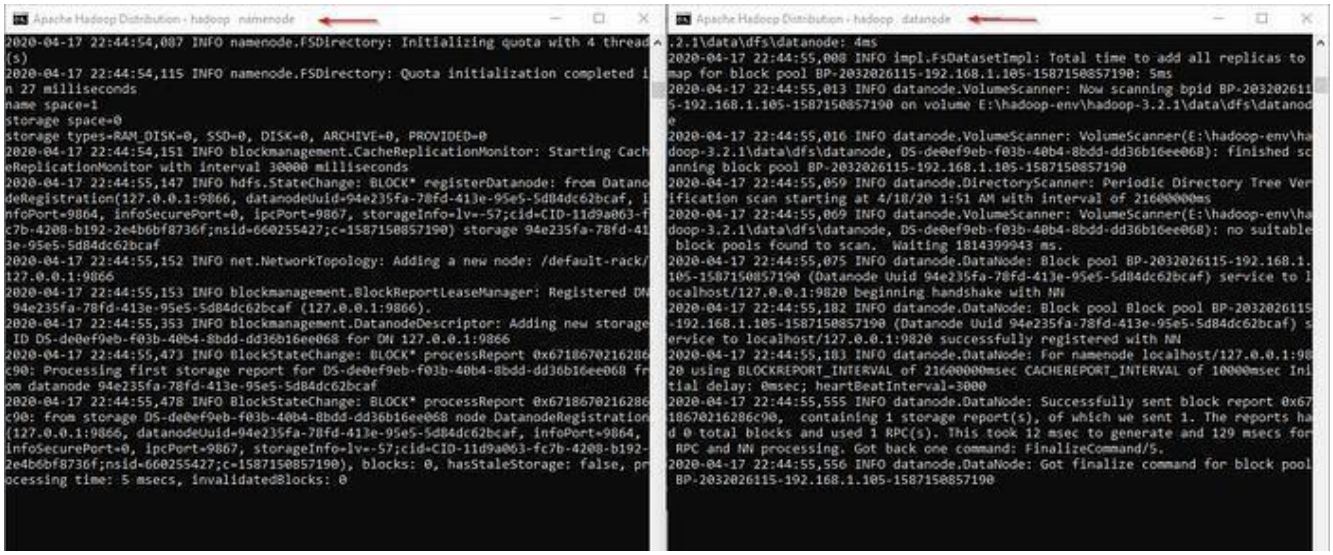


Figure 18 — Hadoop nodes command prompt windows

Next, we must start the Hadoop Yarn service using the following command:

```
./start-yarn.cmd
```

```
PS E:\hadoop-env\hadoop-3.2.1\sbin> .\start-yarn.cmd
starting yarn daemons
PS E:\hadoop-env\hadoop-3.2.1\sbin>
```

Figure 19 — Starting Hadoop Yarn services

Two command prompt windows will open (one for the resource manager and one for the node manager) as follows:

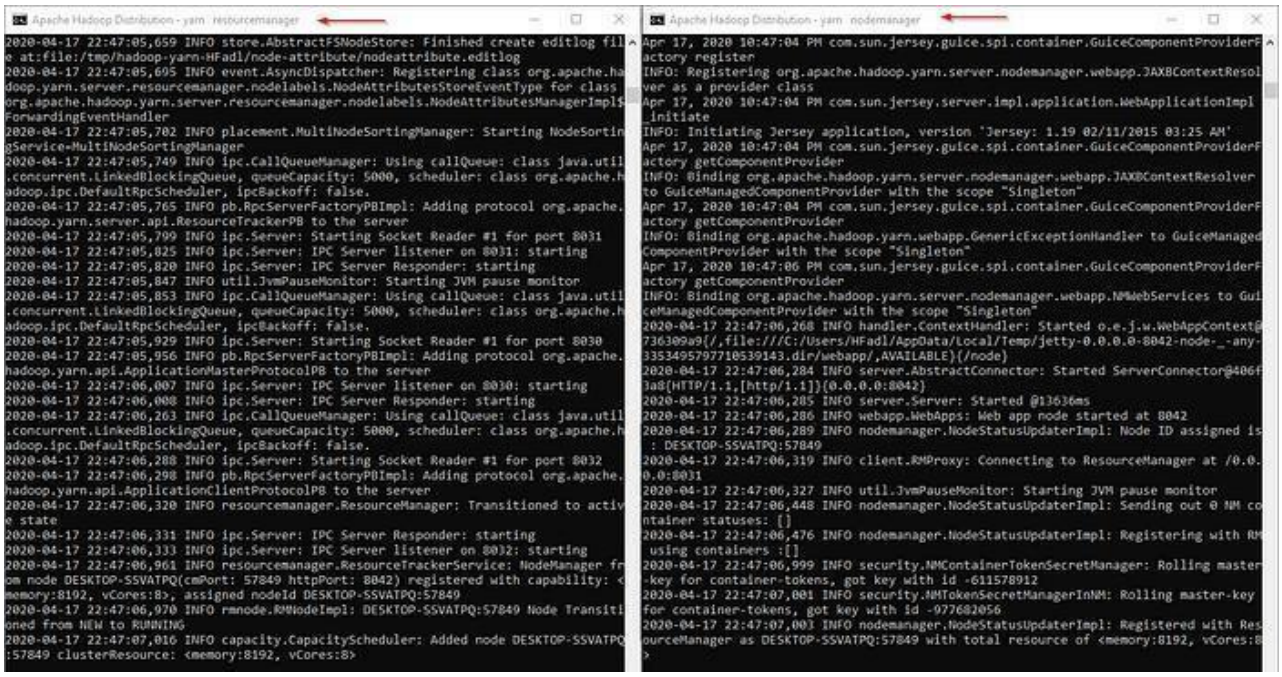


Figure 20— Node manager and Resource manager command prompt windows

Result:

Thus downloading and installing Hadoop &Understanding different Hadoop modes successfully executed.

EXP NO:02 Date:	Hadoop Implementation of file management tasks, such as Adding files and directories, retrieving files and Deleting files

AIM:-

Implement the following file management tasks in Hadoop:
Adding files and directories
Retrieving files
Deleting Files

DESCRIPTION:-

HDFS is a scalable distributed filesystem designed to scale to petabytes ofdata while running on top of the underlying filesystem of the operating system.HDFS keeps track of where the data resides in a network by associating the nameof its rack (or network switch) with the dataset.
Efficientlyscheduletaskstothosenodesthatcontaindata,orwhicharenearesttoit,optimizing bandwidth utilization. Hadoop provides a set of command line utilitiesthatworksimilarlytotheLinuxfilecommands,andserveasyourprimaryinterface with HDFS. We_re going to have a look into HDFS by interacting with itfromthecommandline.
Wewilltakealookatthemostcommonfilemanagementtasksin Hadoop,whichinclude:
Adding filesanddirectoriestoHDFS
RetrievingfilesfromHDFStolocalfilesystem
DeletingfilesfromHDFS

ALGORITHM:-

SYNTAXAND COMMANDSTOADD,RETRIEVEANDDELETEDATAFROMHDFS

Step-1:AddingFilesandDirectoriestoHDFS

Before you can run Hadoop programs on datastored in HDFS, you_ll needto put the data into HDFS first. Let_s create a directory and put a file in it. HDFShas a default working directory of /user/$USER,

where $USER is your login username. This directory isn_t automatically created for you, though, so let_s create itwith themkdir command. For the purpose of illustration, we use chuck.

Youshouldsubstituteyourusernameintheexamplecommands.

hadoopfs-mkdir/user/chuck hadoopfs-putexample.txt
hadoopfs-putexample.txt/user/chuck
Step-2:RetrievingFilesfromHDFS

TheHadoopcommandgetcopiesfilesfromHDFSbacktothelocalfilesystem.Toretrieveex ample.txt,wecanrunthefollowingcommand.

hadoopfs-catexample.txt

Step-3:DeletingFilesfromHDFS

hadoopfs-rmexample.txt

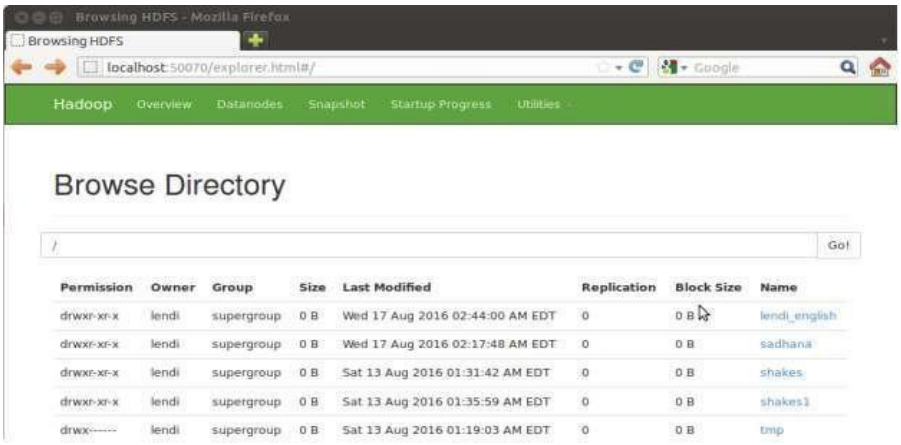
- Commandforcreatingadirectoryinhdfsis**"hdfsdfs–mkdir/lendicse"**
- Addingdirectoryisdonethroughthecommand **"hdfsdfs–putlendi_english/"**.

Step-4:CopyingDatafromNFSto HDFS

Copyingfromdirectorycommandis **"hdfsdfs–copyFromLocal/home/lendi/Desktop/shakes/ glossary/lendicse/"**

- Viewthefilebyusingthecommand**"hdfsdfs–cat/lendi_english/glossary"**
- CommandforlistingofitemsinHadoopis**"hdfsdfs–lshdfs://localhost:9000/"**
- CommandforDeleting filesis**"hdfsdfs–rmr/kartheek"**

Output:



Result:

Thus the Installing of Hadoop in three operating modes has been successfully completed.

EXP NO:03 Date:	Implement of Matrix Multiplication with Hadoop Map Reduce

Aim:

To Implement of Matrix Multiplication with Hadoop Map Reduce

Algorithm:

1. Initialize the input matrices `matrix_A` and `matrix_B`.

2. Check if the number of columns in `matrix_A` is equal to the number of rows in `matrix_B`. If not, raise a `ValueError`.

3. Prepare the input data for the MapReduce simulation:

- Create a list `input_data` that contains strings representing each element in the input matrices in the format: `"matrix,i,j,value"`.

- Each element in `input_data` represents an element in `matrix_A` or `matrix_B`, where `i` is the row index, `j` is the column index, and `value` is the element value.

4. Define the Mapper function:

- The `mapper` function takes the `input_data` list as input.

- For each element in `input_data`, split it into `matrix`, `i`, `j`, and `value`.

- If the element belongs to `matrix_A`, emit intermediate key-value pairs for all possible columns `k` in `matrix_B`: `"i,k,A,j,value"`.

- If the element belongs to `matrix_B`, emit intermediate key-value pairs for all possible rows `k` in `matrix_A`: `"k,j,B,i,value"`.

- Yield each intermediate key-value pair.

5. Define the Reducer function:

- The `reducer` function takes the mapped data as input.

- Initialize an empty `result_matrix` with dimensions `rows_A` x `cols_B`, filled with zeros.

- For each element in the mapped data, split it into `i`, `j`, `matrix_id`, `index`, and `value`.

- If the element belongs to `matrix_A`, compute the partial product for the corresponding row and column in `result_matrix`.
- If the element belongs to `matrix_B`, compute the partial product for the corresponding row and column in `result_matrix`.
- Yield each partial product element.

6. Execute the MapReduce simulation:

- Call the `mapper` function with the `input_data` list as input to obtain the mapped data.

- Call the `reducer` function with the mapped data as input to obtain the reduced data.

7. Collect the results:

- Create an empty list `results`.

- For each element in the reduced data, split it into `i`, `j`, and `value`.

- Append the `(i, j, value)` tuple to the `results` list.

8. Prepare the results matrix:

- Create an empty `result_matrix` with dimensions `rows_A` x `cols_B`, filled with zeros.

- For each element in `results`, set the corresponding element in `result_matrix` to the value.

9. Print the result:

- Print each row in `result_matrix` to display the final matrix multiplication result.

Program:

```
import json


# Input matrices in Python lists format (Replace these with your actual matrices)
matrix_A = [
    [1, 2],


    [3, 4]


]

matrix_B = [
    [5, 6],
    [7, 8]


]


 rows_A, cols_A = len(matrix_A), len(matrix_A[0]) rows_B, cols_B = len(matrix_B), len(matrix_B[0])
 if cols_A != rows_B:
    raise ValueError("Number of columns in matrix A must be equal to the number of rows in
matrix B.")

 # Prepare the input for MapReduce input_data = [      f'A,{i},{j},{value}' for i, row in
enumerate(matrix_A) for j, value in enumerate(row)


] + [      f'B,{i},{j},{value}' for i, row in enumerate(matrix_B) for j, value in
enumerate(row)
]
# Mapper script for matrix multiplication def mapper(input_data):      for line in input_data:
        matrix, i, j, value = line.strip().split(',')
if matrix == 'A':
            for k in range(cols_B):


                yield
f"{i},{k},{matrix},{j},{value}"           elif
matrix == 'B':              for k in range(cols_A):
                yield f"{k},{j},{matrix},{i},{value}"


 # Reducer script for matrix multiplication def reducer(mapped_data):
```

```
    result_matrix = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
for line in mapped_data:
        i, j, matrix_id, index, value =
line.strip().split(',')          i, j, value = int(i), int(j),
float(value)          if matrix_id == 'A':
            result_matrix[i][j] += value * matrix_B[j][int(index)]
elif matrix_id == 'B':
            result_matrix[int(index)][j] += matrix_A[i][int(index)] * value
    for i, row in enumerate(result_matrix):
        for j, value in enumerate(row):
yield f"{i},{j},{value}"



# Execute the MapReduce simulation
mapped_data = mapper(input_data)
reduced_data = reducer(mapped_data) # Collect
the results results = [line for line in
reduced_data]



# Prepare the results matrix result_matrix = [[0 for _ in
range(cols_B)] for _ in range(rows_A)] for line in results:
    i, j, value = line.strip().split(',')
result_matrix[int(i)][int(j)] = float(value)



# Print the result for
row in result_matrix:
    print(row)
```

Output:

```
 [37.0, 47.0]
[81.0, 101.0]
```

Result:

Thus matrix multiplication using map reduce executed successfully.

EXP NO:04 Date:	Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm

Aim:

To run a basic Word Count Map Reduce program to understand Map Reduce Paradigm

Algorithm:

1. Initialize the input text data `text_data` (Replace this with your own text data).

2. Define the `mapper` function:

- The `mapper` function takes the `text_data` as input.

- Split the `text_data` into words using whitespace as the delimiter.

- For each word in the `text_data`, create a key-value pair where the word is the key, and the value is 1 (indicating the occurrence of the word).
- Return a list of these key-value pairs.

3. Define the `reducer` function:

- The `reducer` function takes the mapped data as input.

- Create an empty dictionary `word_count_dict` to store the word counts.

- For each key-value pair in the mapped data:

- If the word is already in `word_count_dict`, increment the count by the value.

- Otherwise, add the word to `word_count_dict` with the count value.

- Return a list of tuples containing the word and its corresponding count from the `word_count_dict`.

4. Execute the Map phase (Mapper):

- Call the `mapper` function with the `text_data` as input to obtain the mapped data.

5. Execute the Reduce phase (Reducer):

- Call the `reducer` function with the mapped data as input to obtain the reduced data.

6. Print the word counts:

- For each word and its count in the reduced data, print the word and count in the format: "{word}: {count}".

Program:

```
# Input text data (Replace this with your own text data) text_data
= '''
MapReduce is a programming paradigm for processing and generating large datasets with a parallel, distributed algorithm on a cluster.

The MapReduce algorithm contains two important tasks, namely Map and Reduce.

Map takes a set of data and converts it into another set of data, where individual elements are broken down into key-value pairs.

The Reduce task takes the output of the Map as an input and combines those data tuples into a smaller set of tuples.

''' def mapper(text_data):

    words = text_data.strip().split()
word_count_pairs = [(word, 1) for word in words]
return word_count_pairs



def reducer(mapped_data):
word_count_dict = {}      for word,
count in mapped_data:         if
word in word_count_dict:
            word_count_dict[word] += count
        else:
            word_count_dict[word] = count
return word_count_dict.items()
# Execute the Map phase (Mapper) mapped_data = mapper(text_data)

# Execute the Reduce phase (Reducer) reduced_data = reducer(mapped_data)

 # Print the word counts for word, count in reduced_data:      print(f"{word}: {count}")
```

Output:

MapReduce: 2

is: 1 a:
5
programming: 1
paradigm: 1
for: 1
processing: 1
and: 4
generating: 1
large: 1
datasets: 1
with: 1
parallel,: 1
distributed: 1
algorithm: 2
on: 1 cluster.:
1
The: 2
contains: 1
two: 1
important: 1
tasks,: 1
namely: 1
Map: 3
Reduce.: 1
takes: 2 set:
3 of: 4
data: 2
converts: 1
it: 1 into: 3
another: 1
data,: 1
where: 1
individual: 1
elements: 1
are: 1
broken: 1
down: 1
key-value: 1
pairs.: 1
Reduce: 1
task: 1 the:
2
output: 1
as: 1 an: 1
input: 1
combines: 1
those: 1
tuples: 1
smaller: 1
tuples.: 1

Result:

Thus a basic Word Count Map Reduce program was executed successfully.

EXP NO:05 Date:	Installation of Hive along with practice examples.

Aim:

To Install Hive tool

Procedure:

1.1. 7zip

In order to extract tar.gz archives, you should install the 7zip tool.

1.2. Installing Hadoop

To install Apache Hive, you must have a Hadoop Cluster installed and running: You can refer to our previously published step-by-step guide to install Hadoop 3.2.1 on Windows 10.

1.3. Apache Derby

In addition, Apache Hive requires a relational database to create its Metastore (where all metadata will be stored). In this guide, we will use the Apache Derby database 4.

Since we have Java 8 installed, we must install Apache Derby 10.14.2.0 version (check downloads page) which can be downloaded from the following link.

Once downloaded, we must extract twice *(using 7zip: the first time we extract the .tar.gz file, the second time we extract the .tar file)* the content of the db-derby-10.14.2.0-bin.tar.gz archive into the desired installation directory. Since in the previous guide we have installed Hadoop within —E:\hadoop-env\hadoop-3.2.1\‖ directory, we will extract Derby into —E:\hadoop-env\db-derby-10.14.2.0\‖ directory.

1.4. Cygwin

Since there are some Hive 3.1.2 tools that aren‘t compatible with Windows (such as schematool). We will need the Cygwin tool to run some Linux commands.

2. Downloading Apache Hive binaries

In order to download Apache Hive binaries, you should go to the following website: https://downloads.apache.org/hive/hive-3.1.2/. Then, download the apache-hive3.1.2.-bin.tar.gz file.

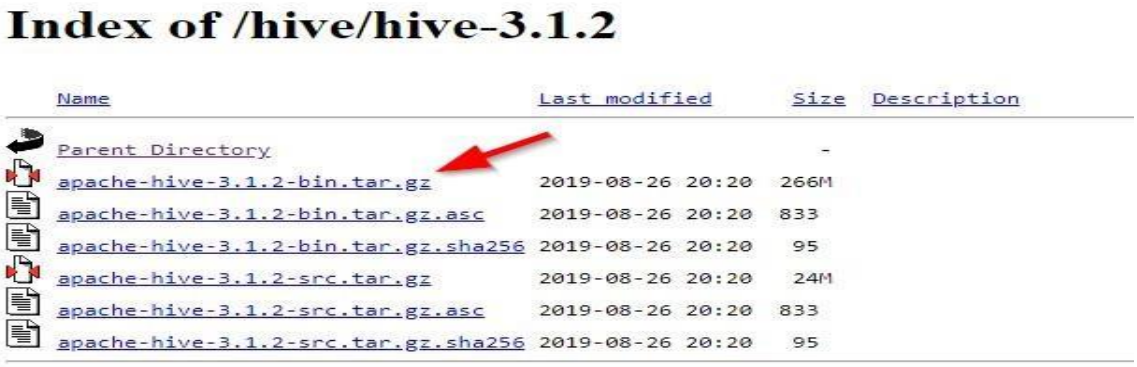


Figure 1 — apache-hive.3.1.2-bin.tar.gz file

When the file download is complete, we should extract twice *(as mentioned above)* the apache-hive.3.1.2-bin.tar.gz archive into ―E:\hadoop-env\apache-hive-3.1.2‖ directory (Since we decided to use E:\hadoop-env\‖ as the installation directory for all technologies used in the previous guide.

3. Setting environment variables

After extracting Derby and Hive archives, we should go to Control Panel > System and Security > System. Then Click on ―Advanced system settings‖.

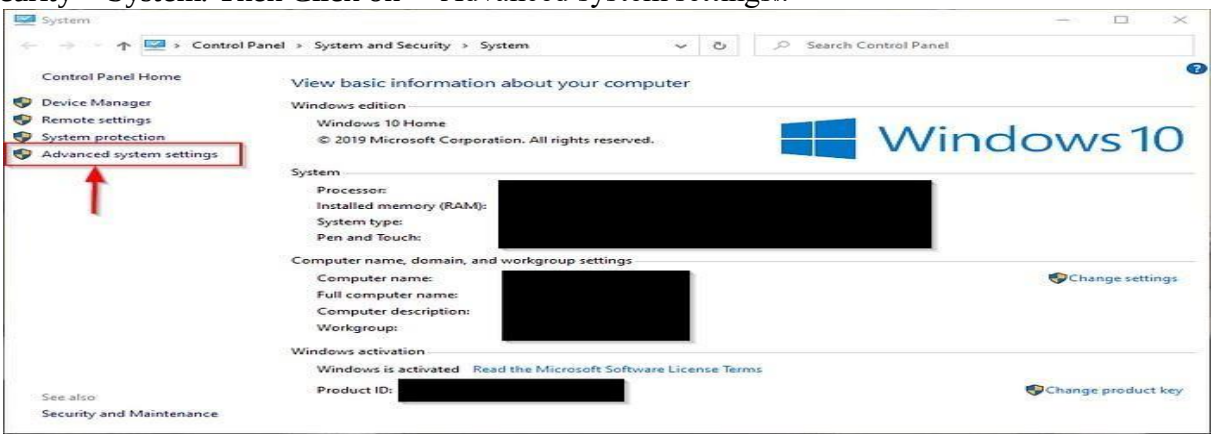


Figure 2 — Advanced system settings

In the advanced system settings dialog, click on ―Environment variables‖ button.

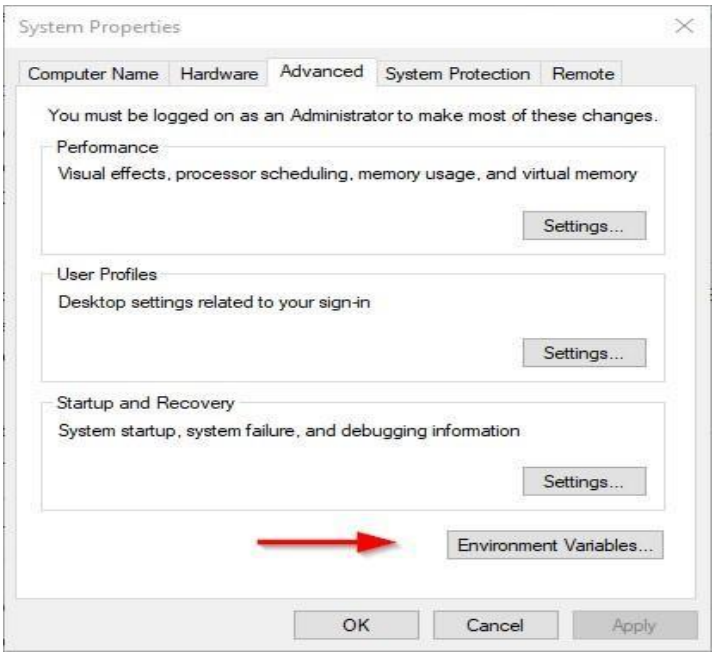


Figure 3 — Opening environment variables editor Now we should add the following user variables:

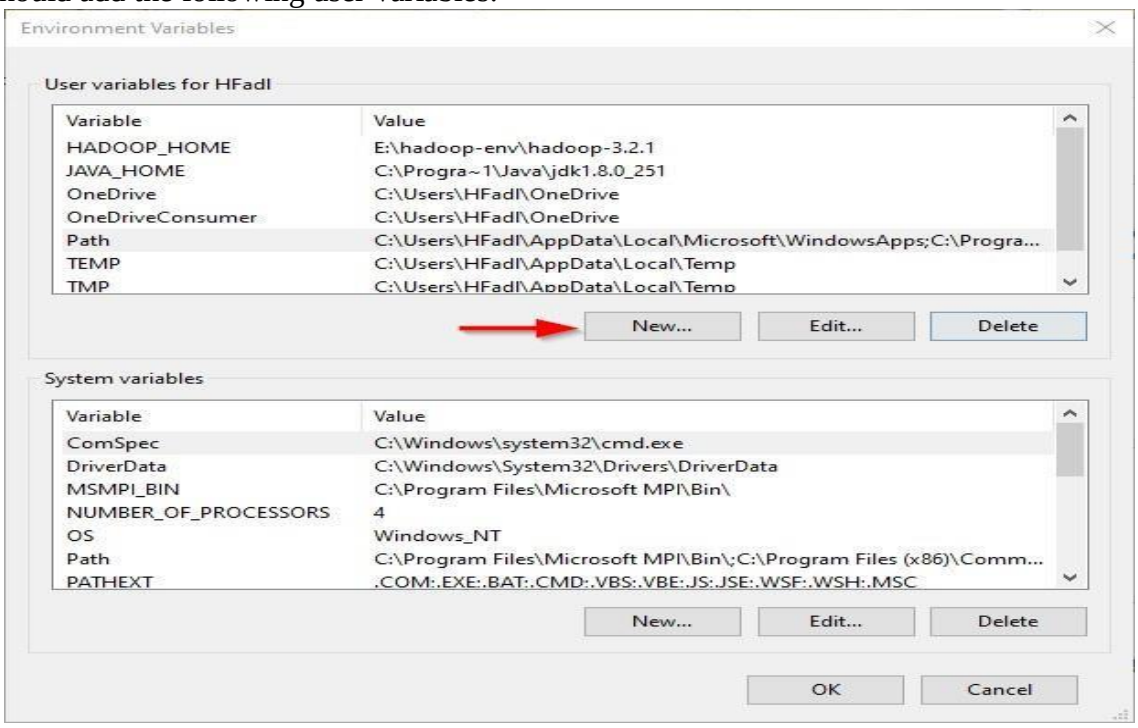


Figure 4 — Adding User variables

- HIVE_HOME: —E:\hadoop-env\apache-hive-3.1.2\‖
- DERBY_HOME: —E:\hadoop-env\db-derby-10.14.2.0\‖
- HIVE_LIB: —%HIVE_HOME%\lib‖

- HIVE_BIN: —%HIVE_HOME%\bin‖
- HADOOP_USER_CLASSPATH_FIRST: —true‖



Figure 5 — Adding HIVE_HOME user variable

Besides, we should add the following system variable:

- HADOOP_USER_CLASSPATH_FIRST: —true‖

Now, we should edit the Path user variable to add the following paths:

- %HIVE_BIN%
- %DERBY_HOME%\bin

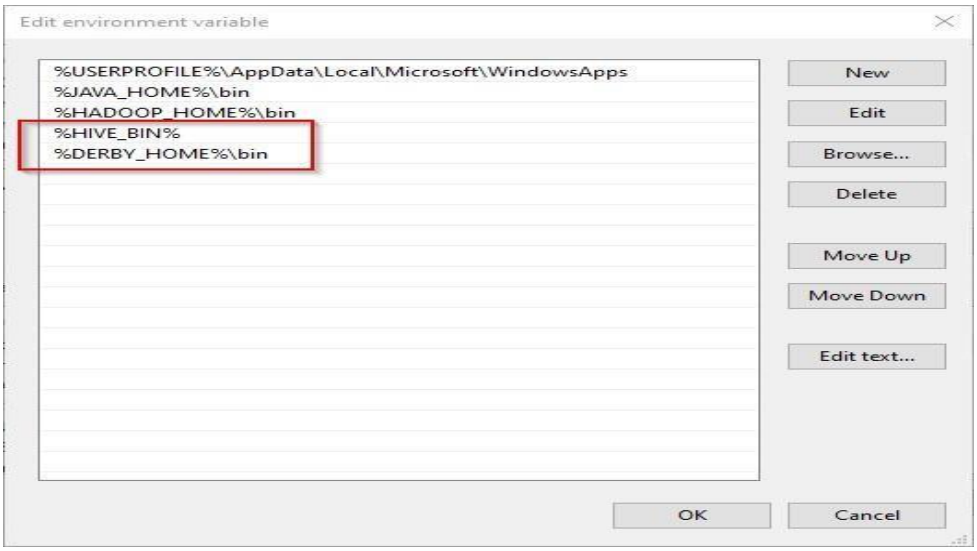


Figure 6 — Editing path environment variable

4. Configuring Hive

4.1. Copy Derby libraries

Now, we should go to the Derby libraries directory (E:\hadoop-env\db-derby-10.14.2.0\lib) and copy all *.jar files.

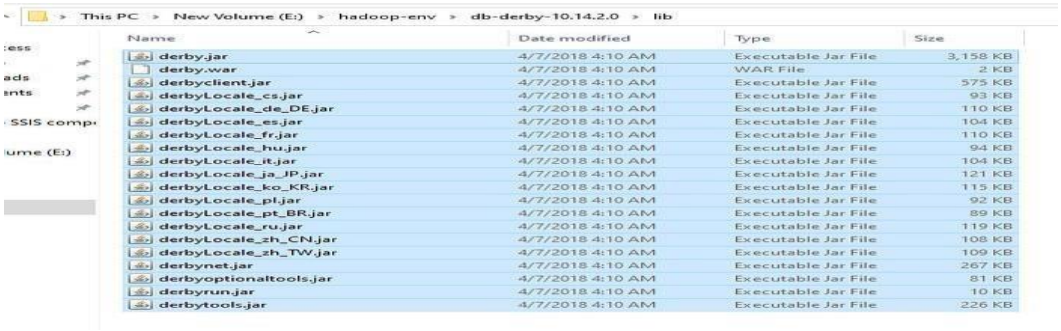


Figure 7 — Copy Derby libraries

Then, we should paste them within the Hive libraries directory (E:\hadoop-env\apache-hive-3.1.2\lib).

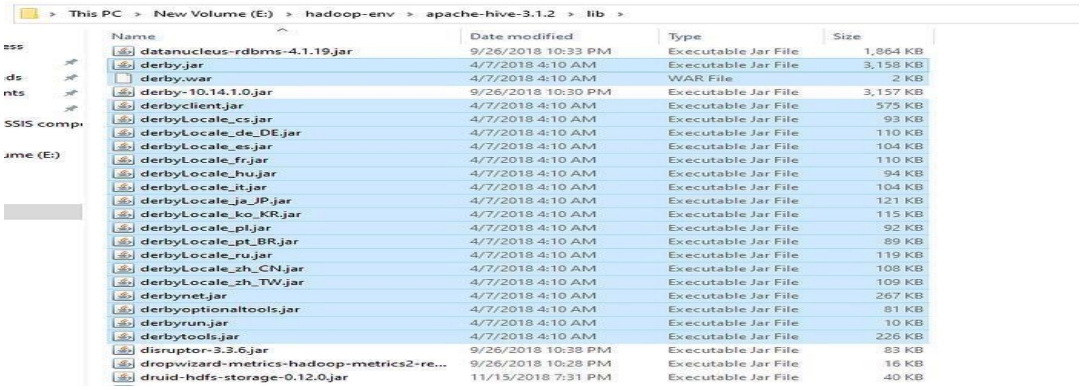


Figure 8 — Paste Derby libraries within Hive libraries directory

4.2. Configuring hive-site.xml

Now, we should go to the Apache Hive configuration directory (E:\hadoop-env\apache-hive-

3.1.2\conf) create a new file —hive-site.xml‖. We should paste the following XML code within this file:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration><property> <name>javax.jdo.option.ConnectionURL</name>
<value>jdbc:derby://localhost:1527/metastore_db;create=true</value>
<description>JDBC connect string for a JDBC metastore</description>
</property><property>
<name>javax.jdo.option.ConnectionDriverName</name>
<value>org.apache.derby.jdbc.ClientDriver</value>
<description>Driver class name for a JDBC
metastore</description> </property> <property>
<name>hive.server2.enable.doAs</name>
<description>Enable user impersonation for HiveServer2</description>

<value>true</value>
</property>
<property>
<name>hive.server2.authentication</name>
<value>NONE</value>
<description> Client authentication types. NONE: no authentication check LDAP: LDAP/AD
based authentication KERBEROS: Kerberos/GSSAPI authentication CUSTOM: Custom
authentication provider (Use with property hive.server2.custom.authentication.class)
</description>
</property>
<property>
<name>datanucleus.autoCreateTables</nam>e
<value>True</value>
</property>
</configuration>
```

5. Starting Services

5.1. Hadoop Services

To start Apache Hive, open the command prompt utility as administrator. Then, start the Hadoop services using start-dfs and start-yarn commands (as illustrated in the Hadoop installation guide).

5.2. Derby Network Server

Then, we should start the Derby network server on the localhost using the following command:

```
E:\hadoop-env\db-derby-10.14.2.0\bin\StartNetworkServer-h  0.0.0.0
```

6. Starting Apache Hive

Now, let try to open a command prompt tool and go to the Hive binaries directory (E:\hadoopenv\apache-hive-3.1.2\bin) and execute the following command:

```
hive
```

We will receive the following error:

```
'hive' is not recognized as an internal or external command, operable program or batch file.
```

This error is thrown since the Hive 3.x version is not built for Windows (only in some Hive 2.x versions). To get things working, we should download the necessary *.cmd files from the following link: https://svn.apache.org/repos/asf/hive/trunk/bin/. Note that, you should keep the folder hierarchy (bin\ext\util).

You can download all *.cmd files from the following GitHub repository

- https://github.com/HadiFadl/Hive-cmd

Now if we try to execute the ―hive‖ command, we will receive the following error: Exception in thread "main" java.lang.NoSuchMethodError:

```
com.google.common.base.Preconditions.checkArgument(ZLjava/lang/String;Ljava/lang/Obj
ect;)
V
at org.apache.hadoop.conf.Configuration.set(Configuration.java:1357) at
org.apache.hadoop.conf.Configuration.set(Configuration.java:1338) at
org.apache.hadoop.mapred.JobConf.setJar(JobConf.java:518) at
org.apache.hadoop.mapred.JobConf.setJarByClass(JobConf.java:536) at
org.apache.hadoop.mapred.JobConf.<init>(JobConf.java:430) at
org.apache.hadoop.hive.conf.HiveConf.initialize(HiveConf.java:5141) at
org.apache.hadoop.hive.conf.HiveConf.<init>(HiveConf.java:5104) at
org.apache.hive.beeline.HiveSchemaTool.<init>(HiveSchemaTool.java:96) at
org.apache.hive.beeline.HiveSchemaTool.main(HiveSchemaTool.java:1473) at
sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method) at
sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62) at
sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:4
3) at java.lang.reflect.Method.invoke(Method.java:498) at
org.apache.hadoop.util.RunJar.run(RunJar.java:318) at
org.apache.hadoop.util.RunJar.main(RunJar.java:232)
```

This error is thrown due to a Bug mentioned in the following Hive issue link: HIVE-22718.

As mentioned in the comments, this issue can be solved by replacing the guava-19.0.jar stored in ―E:\hadoop-env\apache-hive-3.1.2\lib‖ with Hadoop‘s guava-27.0-jre.jar found in ―E:\hadoop-env\hadoop-3.2.1\share\hadoop\hdfs\lib‖.

Note: This file is also uploaded to the GitHub repository mentioned above.

Now, if we run hive command again, then Apache Hive will start successfully.

7. Initializing Hive

After ensuring that the Apache Hive started successfully. We may not be able to run any HiveQL command. This is because the Metastore is not initialized yet. Besides HiveServer2 service must be running.

To initialize Metastore, we need to use schematool utility which is not compatible with windows. To solve this problem, we will use Cygwin utility which allows executing Linux command from windows.

7.1. Creating symbolic links

First, we need to create the following directories:

- E:\cygdrive
- C:\cygdrive

Now, open the command prompt as administrator and execute the following commands:

```
mklink /J E:\cygdrive\e\ E:\
mklink /J  C:\cygdrive\c\ C:\
```

These symbolic links are needed to work with Cygwin utility properly since Java maycause some problems.

7.2. Initializing Hive Metastore

Open Cygwin utility and execute the following commands to define the environment variables:

```
export HADOOP_HOME='/cygdrive/e/hadoop-env/hadoop-3.2.1'
export PATH=$PATH:$HADOOP_HOME/bin
export  HIVE_HOME='/cygdrive/e/hadoop-env/apache-hive-3.1.2'
export PATH=$PATH:$HIVE_HOME/bin export
HADOOP_CLASSPATH=$HADOOP_CLASSPATH:$HIVE_HOME/lib/*.jar
```

We can add these lines to the —~/.bashrc‖ file then you don‘t need to write them each time you open Cygwin.

Now, we should use the schematool utility to initialize the Metastore:

```
$HIVE_HOME/bin/schematool -dbType derby -initSchema
```

7.3. Starting HiveServer2 service

Now, open a command prompt and run the following command:

```
hive --service hiveserver2 start
```

We should leave this command prompt open, and open a new one where we should start Apache Hive using the following command:

```
hive
```

7.4. Starting WebHCat Service (Optional)

In the project we are working on, we need to execute HiveQL statement from SQL Server Integration Services which can access Hive from the WebHCat server. To start the WebHCat server, we should open the Cygwin utility and execute the following command:

```
$HIVE_HOME/hcatalog/sbin/webhcat_server.sh  start
```

Output:

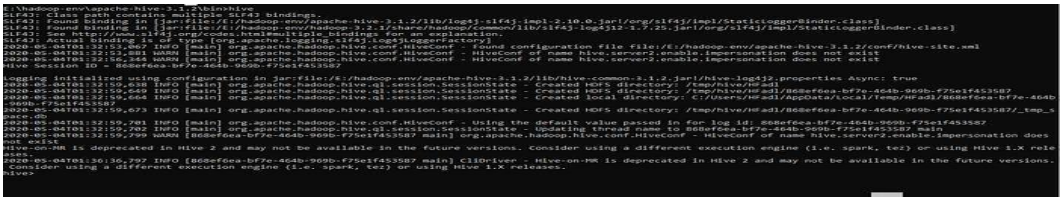


Figure 9 — Starting Apache Hive

Result:

Thus the installation of hive successfully executed.

EXP NO:06 Date:	Installation of HBase, Installing thrift along with Practice examples

Aim:

To install Hbase and thrift

Procedure:

Prerequisite

- Install Java JDK - You can download it from this link.
  (https://www.oracle.com/java/technologies/downloads/)
  The Java Development Kit (JDK) is a cross-platform software development environment that includes tools and libraries for creating Java-based software applications and applets.

- Download Hbase - Download Apache Hbase from this link.
  (https://hbase.apache.org/downloads.html)

Must Read Apache Server

Steps

Step-1 (Extraction of files)

Extract all the files in C drive

Step-2 (Creating Folder)

Create folders named "hbase" and "zookeeper."

Step-3 (Deleting line in HBase.cmd)

Open hbase.cmd in any text editor.

Search for line **%HEAP_SETTINGS% and remove it.**

Step-4 (Add lines in hbase-env.cmd)

Now open hbase-env.cmd, which is in the conf folder in any text editor.

Add the below lines in the file after the comment session.

```
set JAVA_HOME=%JAVA_HOME%
set HBASE_CLASSPATH=%HBASE_HOME%\lib\client-facing-thirdparty\* set
HBASE_HEAPSIZE=8000
set HBASE_OPTS="-XX:+UseConcMarkSweepGC" "-Djava.net.preferIPv4Stack=true" set
SERVER_GC_OPTS="-verbose:gc" "-XX:+PrintGCDetails" "-XX:+PrintGCDateStamps"
%HBASE_GC_OPTS%
set HBASE_USE_GC_LOGFILE=true
```

```
set HBASE_JMX_BASE="-Dcom.sun.management.jmxremote.ssl=false"
"Dcom.sun.management.jmxremote.authenticate=false"

set HBASE_MASTER_OPTS=%HBASE_JMX_BASE% "-
Dcom.sun.management.jmxremote.port=10101"
set HBASE_REGIONSERVER_OPTS=%HBASE_JMX_BASE% "-
Dcom.sun.management.jmxremote.port=10102" set
HBASE_THRIFT_OPTS=%HBASE_JMX_BASE%
"Dcom.sun.management.jmxremote.port=10103"
set HBASE_ZOOKEEPER_OPTS=%HBASE_JMX_BASE% -
Dcom.sun.management.jmxremote.port=10104"
set
HBASE_REGIONSERVERS=%HBASE_HOME%\conf\regionservers
set HBASE_LOG_DIR=%HBASE_HOME%\logs set
HBASE_IDENT_STRING=%USERNAME% set
HBASE_MANAGES_ZK=true
```

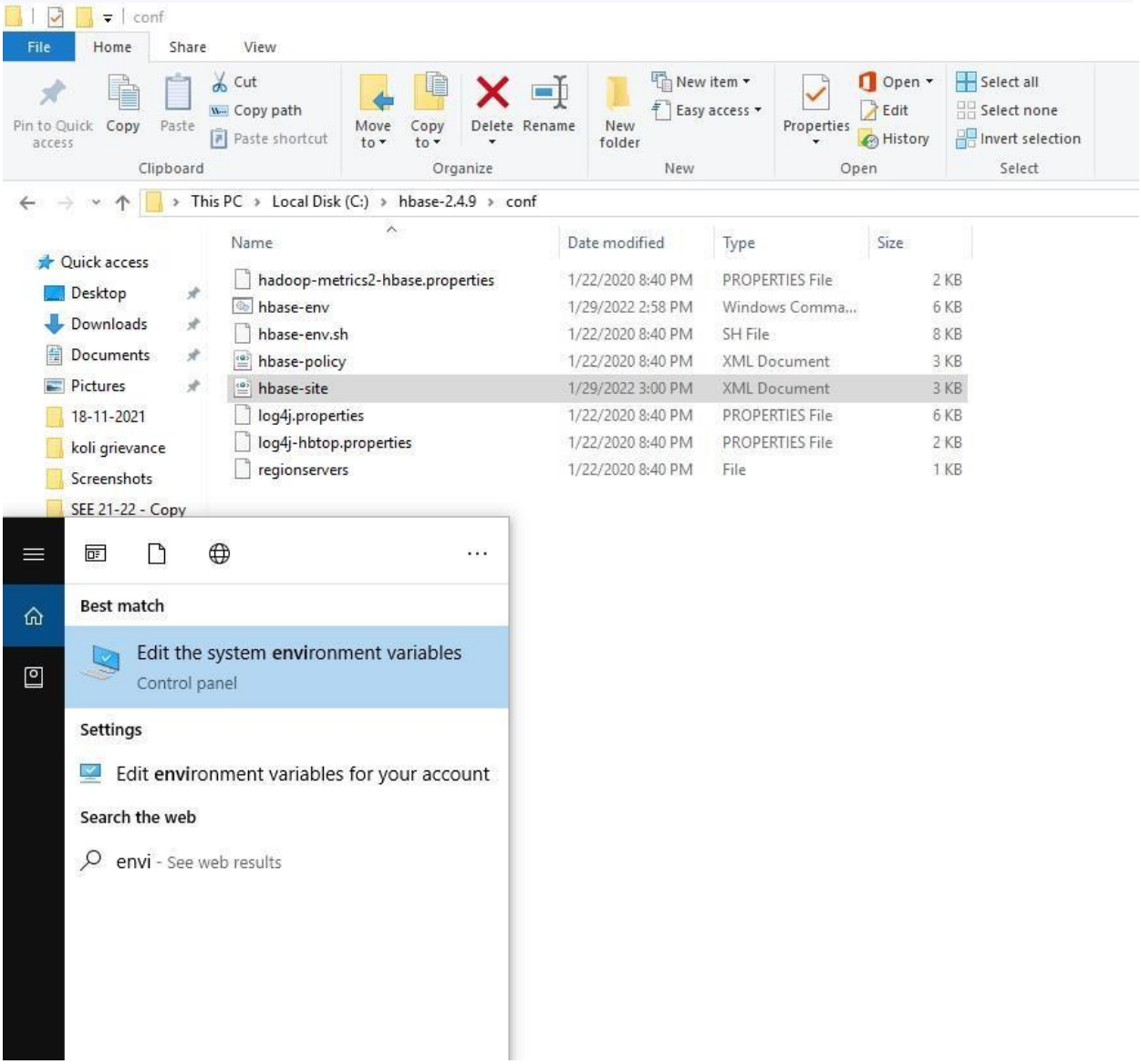
Step-5 (Add the line in Hbase-site.xml)
Open hbase-site.xml, which is in the conf folder in any text editor.
Add the lines inside the <configuration> tag.
A distributed HBase entirely relies on Zookeeper (for cluster configuration and management). ZooKeeper coordinates, communicates and distributes state between the Masters and RegionServers in Apache HBase. HBase's design strategy is to use ZooKeeper solely for transient data (that is, for coordination and state communication). Thus, removing HBase's ZooKeeper data affects only temporary operations – data can continue to be written and retrieved to/from HBase.

```
<property>
   <name>hbase.rootdir</name>
   <value>file:///C:/Documents/hbase-2.2.5/hbase</value>
</property>
<property>
   <name>hbase.zookeeper.property.dataDir</name>
   <value>/C:/Documents/hbase-2.2.5/zookeeper</value>
</property>
<property>
    <name> hbase.zookeeper.quorum</name>
   <value>localhost</value>
</property>
```

Step-6 (Setting Environment Variables)
Now set up the environment variables.
Search "System environment variables." Now click on " Environment Variables." Then click on "New."

Variable name: HBASE_HOME
Variable Value: Put the path of the Hbase folder.
We have completed the HBase Setup on Windows 10 procedure.

Output:



Result:

Thus installation of Hbase & thrift executed successfully

EXP NO:07 Date:	Practice importing and exporting data from various databases.Software Requirements: Cassandra, Hadoop, Java, Pig, Hive and HBase.

Aim

To Practice importing and exporting data from various databases

Procedure:

To practice importing and exporting data from various databases, you'll need to set up the required software and databases on your system. Below are the software requirements and steps to get started with importing and exporting data from each database:

1. Cassandra:

- Install Cassandra on your system. You can download it from the Apache Cassandra website and follow the installation instructions for your operating system.
- Start the Cassandra server using the `cassandra -f` command.
- Use the `cqlsh` command-line tool to interact with Cassandra and create keyspaces, tables, and perform data operations.

2. Hadoop and HDFS:

- Install Hadoop on your system. You can download it from the Apache Hadoop website and follow the installation instructions for your operating system.
- Start the HDFS service using the `start-dfs.sh` command.
- Use the `hadoop fs` command-line tool to interact with HDFS and perform file operations.

3. Java:

- Install Java Development Kit (JDK) on your system. You can download it from the Oracle or OpenJDK website and follow the installation instructions for your operating system.

4. Pig:

- Install Pig on your system. You can download it from the Apache Pig website and follow the installation instructions for your operating system.
- Start Pig using the `pig` command in the terminal.

5. Hive:

- Install Hive on your system. You can download it from the Apache Hive website and follow the installation instructions for your operating system.
- Start the Hive service using the `hive` command in the terminal.

6. HBase:

- Install HBase on your system. You can download it from the Apache HBase website and follow the installation instructions for your operating system.
- Start the HBase service using the `start-hbase.sh` command.

Once you have set up the required software and services, you can start practicing importing and exporting data from these databases. Here are some common tasks to practice:

- Import data into Cassandra from CSV files or other data sources using `cqlsh` or Python's `cassandra-driver`.
- Export data from Cassandra to CSV files or other formats using `cqlsh` or Python's `cassandra-driver`.
- Import data into HDFS from local files or other data sources using the `hadoop fs` command.
- Export data from HDFS to local files or other formats using the `hadoop fs` command.
- Use Pig to perform data transformations and analysis on data stored in HDFS.
- Create Hive tables to manage structured data and perform SQL-like queries on the data.
- Import data into Hive tables from CSV files or other data sources.
- Export data from Hive tables to CSV files or other formats.

- Import data into HBase from CSV files or other data sources using Java or Python.

- Export data from HBase to CSV files or other formats using Java or Python.

Program:

```
# Install cassandra-driver library !pip
install cassandra-driver from
cassandra.cluster import Cluster

# Connect to Cassandra cluster
cluster = Cluster(['localhost']) session
= cluster.connect()

# Create a keyspace and table

session.execute("CREATE KEYSPACE IF NOT EXISTS test WITH REPLICATION =

{'class': 'SimpleStrategy', 'replication_factor': 1}")

session.execute("CREATE TABLE IF NOT EXISTS test.users (id UUID PRIMARY KEY,
name TEXT, age INT)")

# Insert data into the table

session.execute("INSERT INTO test.users (id, name, age) VALUES (uuid(), 'John Doe',
30)")

session.execute("INSERT INTO test.users (id, name, age) VALUES (uuid(), 'Jane Smith',
25)")


# Query data from the table rows =
session.execute("SELECT * FROM test.users") for
row in rows:
    print(row.id, row.name, row.age)


# Close the connection

session.shutdown() cluster.shutdown()
# Install hdfs library !pip install
hdfs from hdfs import
InsecureClient
```

```
# Connect to HDFS hdfs_client = InsecureClient('http://localhost:50070',
user='your_username')


# Upload a file to HDFS hdfs_client.upload('/user/your_username/data',
'local_file.txt')


# Download a file from HDFS hdfs_client.download('/user/your_username/data/local_file.txt',
'downloaded_file.txt')


# List files in a directory on HDFS files =
hdfs_client.list('/user/your_username/data')
print(files)
# Install pyhive library

!pip install pyhive

from pyhive import hive

# Connect to Hive conn = hive.connect(host='localhost', port=10000, auth='NONE',
database='default')

# Create a table in Hive with
conn.cursor() as cursor:


    cursor.execute("CREATE TABLE IF NOT EXISTS test_table (id INT, name STRING,
age INT)")

# Insert data into the table with
conn.cursor() as cursor:
    cursor.execute("INSERT INTO test_table VALUES (1, 'John Doe', 30)")
cursor.execute("INSERT INTO test_table VALUES (2, 'Jane Smith', 25)")

# Query data from the table with conn.cursor() as
cursor:     cursor.execute("SELECT * FROM
test_table")
    results = cursor.fetchall()
for row in results:
print(row)


# Close the connection conn.close()
```

Output:

1. Sample output for importing data into Cassandra:

Assume you have a CSV file named `users.csv` with the following data:

```
id,name,age
1,John Doe,30
2,Jane Smith,25
3,Bob Johnson,40
```

After importing the data into a Cassandra table named `users`, the output of a query to retrieve all users would look like:

```
 id | name        |
age
- - -+--------------+------

 1 |   John Doe |  30

 2 | Jane Smith |  25

 3 | Bob Johnson|  40

```

Sample output for exporting data from Cassandra:

After exporting the `users` table from Cassandra to a CSV file named `users_export.csv`, the contents of the file would look like:

```
id,name,age
1,John Doe,30
2,Jane Smith,25
3,Bob Johnson,40
```

2. Sample output for importing data into HDFS:

Assume you have a local file named `data.txt` with the following data:

```
Hello, world!

This is a sample text.
```

After importing the `data.txt` file into HDFS at the path `/user/your_username/data.txt`, you can view the contents using the `hadoop fs -cat` command:

```
$ hadoop fs -cat /user/your_username/data.txt Hello,
world!
This is a sample text.
```

3. Sample output for exporting data from HDFS:

After exporting the data from HDFS to a local file named `data_export.txt`, the contents of the file would be the same as the original data:

```

Hello, world!

This is a sample text.

```
4.Sample output for using Hive to perform SQL-like queries:

Assume you have a Hive table named `employees` with the following data:

```
id,name,department,salary 1,John
Doe,HR,50000
2,Jane Smith,Engineering,60000
3,Bob Johnson,Finance,55000
```

You can use Hive to run SQL-like queries on this data:

```

-- Hive query to retrieve all employees from the HR department
```

SELECT * FROM employees WHERE department = 'HR';

```

The output would be:

```
1   John Doe   HR 50000
```

1. Sample output for importing and exporting data in HBase:

Assume you have a CSV file named `products.csv` with the following data:

```
product_id,product_name,category,price
101,Widget A,Electronics,25.99
102,Widget B,Electronics,19.99
103,Widget C,Home,12.99
```After importing the data into an HBase table named `products`, you can use the HBase shell to view the contents:

```
hbase(main):001:0> scan 'products'
```
```

ROW	COLUMN+CELL
101	column=category: , timestamp=1650554343510, value=Electronics
101	column=price: , timestamp=1650554343510, value=25.99
101	column=product_name: , timestamp=1650554343510, value=Widget A
102	column=category: , timestamp=1650554343510, value=Electronics
102	column=price: , timestamp=1650554343510, value=19.99
102	column=product_name: , timestamp=1650554343510, value=Widget B
103	column=category: , timestamp=1650554343510, value=Home
103	column=price: , timestamp=1650554343510, value=12.99
103	column=product_name: , timestamp=1650554343510, value=Widget C

3 row(s)

Took 0.0573 seconds

``` After exporting the data from the `products` table in HBase to a CSV file named `products_export.csv`, the contents of the file would look like:

```

product_id,product_name,category,price 101,Widget A,Electronics,25.99
102,Widget B,Electronics,19.99 103,Widget C,Home,12.99 ```

**Result:**

Thus importing and exporting data from various databases executed successfully.

| | |
|---|---|
| EXP NO:08<br>Date: | Content Beyond Syllabus : Develop an Application to Find the Maximum Temperature Using Apache Spark |

**Aim**

To develop a Spark application that finds the maximum temperature recorded for each weather station.

**Objective**

- To understand the basics of Apache Spark.
- To process large datasets using Spark RDDs.
- To determine the maximum temperature for each weather station.

**Software Requirements**

- Apache Spark
- Java JDK 8 or above
- Python 3.x (PySpark)
- Hadoop (optional)
- VS Code / PyCharm / Jupyter Notebook

**Input File (temperature.txt)**

ITE001,2025-01-01,35
ITE001,2025-01-02,38
ITE001,2025-01-03,36
ITE002,2025-01-01,30
ITE002,2025-01-02,32
ITE002,2025-01-03,34
ITE003,2025-01-01,28
ITE003,2025-01-02,31
ITE003,2025-01-03,29

Format:

StationID,Date,Temperature

**Algorithm**

1. Start the Spark Session.
2. Load the temperature dataset.
3. Split each record using a comma delimiter.

4. Extract the Station ID and Temperature.
5. Convert temperature into an integer.
6. Create key-value pairs as (StationID, Temperature).
7. Apply reduceByKey() using the max() function.
8. Display the maximum temperature of each station.
9. Stop the Spark Session.

**PySpark Program**

```
from pyspark import SparkContext

Create Spark Context
sc = SparkContext("local", "Maximum Temperature")

Load input file
lines = sc.textFile("temperature.txt")

Create (StationID, Temperature) pairs
temperature = lines.map(lambda x: x.split(",")) \
                .map(lambda x: (x[0], int(x[2])))

Find maximum temperature
maxTemp = temperature.reduceByKey(lambda x, y: max(x, y))

Display output
print("Maximum Temperature of Each Station")
for station, temp in maxTemp.collect():
    print(station, ":", temp, "°C")

Stop Spark
sc.stop()
```

Output

```
Maximum Temperature of Each Station

ITE001 : 38 °C
ITE002 : 34 °C
ITE003 : 31 °C
```

Explanation

- textFile() reads the dataset.
- split(",") separates the fields.
- map() extracts (StationID, Temperature) pairs.
- reduceByKey(max) compares temperatures belonging to the same station and returns the highest value.
- collect() retrieves the results from the cluster for display.

Result:-

The Apache Spark application was successfully developed and executed to determine the maximum temperature for each weather station using the reduceByKey() transformation. The program efficiently processed the dataset and produced the expected output.